QemuWeb configuration manual

QMS74 · v0.7.0 · JSON schema 4

Scope. This manual describes the JSON files written and read by QemuWeb and the commands generated by its runner. The maintained source is `docs/manual.md`; build the A4 PDF with `./scripts/build-manual.sh`. Old JSON schemas and the engine value `virtualbox` are not accepted.

Quick reference — keywords

`F` means fixed for one host, `E` editable in Host Settings and applied on next start, `S` a session choice, and `G` generated. Braces list allowed values, including singletons. † marks a future or experimental value outside the validated baseline. The session ID is the 16-hex-character runtime directory name.

`session.json`

Keyword	Values / validation	Default	Flag
`schemaVersion`	`{4}`	`4`	F
`engine`	`{qemu, vbox}`	`qemu`	S
`hosts`	unique list of `h-` + 16 hex IDs	`[]`	S
`switches`	unique list of `s-` + 16 hex IDs	`[]`	S
`baseImages`	map of used system IDs to image identity	`{}`	S
`baseImages.<system>.format`	`{raw, vdi}`, selected by engine	—	F
`baseImages.<system>.size`	positive byte count	—	F
`baseImages.<system>.sha256`	64 lowercase hex characters	—	F

config.json

`systems.default` is abstract. Each concrete system uses `basedOn` to inherit it and must resolve a non-null image for both engines. A keyword without suffix applies to both engines. `keyword:qemu` or `keyword:vbox` applies only to that engine. For each level of the parent-to-child chain, QemuWeb applies common keywords first, then that level's matching engine variants. Thus `memory` on a child overrides `memory:qemu` inherited from its parent, while `memory:qemu` on the child overrides its own `memory`. Unknown suffixes, parents and cycles are errors.

Keyword in `systems.<system>`	Values / validation	Default in `systems.default`	Flag
`basedOn`	parent system ID; absent on `default`	—	F
`type`	`{linux}`	`linux`	F
`arch`	`{x86_64}`	`x86_64`	F
`mode`	`{text}`	`text`	F
`label`	nonempty display text	`Default Linux`	F
`memory`	integer 64–32768 MB	`512`	E
`cpus`	integer 1–64	`1`	E
`clock`	`{utc, localtime}`	`utc`	E
`startupDiskEnabled`	`{false, true}`	`false`	E
`firmware`	`{bios}`	`bios`	F
`acceleration:qemu`	`{kvm}`	`kvm`	F
`acceleration:vbox`	`{default}`	`default`	F
`boot`	`{disk}`	`disk`	F
`extraOptions:qemu`	argument array with `-machine q35 -cpu host -nic none`	shown in sample	F
`cdromInterface`	`{ide}`	`ide`	F
`diskInterface:qemu`	`{ide, virtio}`	`virtio`	F
`diskInterface:vbox`	`{sata}`	`sata`	F
`networkModel`	`{virtio, e1000}`	`virtio`	F
`image:qemu`	RAW image path	`null` in abstract base	F
`image:vbox`	VDI image path	`null` in abstract base	F

Every keyword in the table may use an unsuffixed or engine-specific form when its value is valid for each engine to which it resolves. For example, the JSON members `"memory:qemu": 1024` and `"memory:vbox": 2048` give different defaults for the same system. The displayed values and sparse host overrides use the selected engine's resolved profile. `image: null` may remain only in the abstract base; concrete profiles require an image for each engine. Paths are relative to the project root or absolute. QEMU's machine, CPU and implicit NIC suppression come from the fixed `extraOptions:qemu` argument array. VBox's `Linux_64`, `IntelAhci` and `PIIX4` remain runner constants.

Translation of each resolved system keyword

A triple `"<keyword>:<engine>": <value>` contributes only to the named engine. An unsuffixed `"<keyword>": <value>` contributes to both. The table below gives the translation **after inheritance and engine selection**; a host override replaces the resolved value for the four editable keywords. – means no direct runner argument. All entries use the exact argv passed by `server.js`, with generated paths shown as placeholders.

Keyword / selected value	QEMU runner translation	VBox runner translation
`"basedOn": "<system>"`	Resolve parent profile; no CLI argument	Same
`"type": "linux"`	Validate supported guest type	Together with `arch`: `createvm --ostype Linux_64`
`"arch": "x86_64"`	Select `QEMU_SYSTEM_X86_64`	Together with `type`: `createvm --ostype Linux_64`
`"mode": "text"`	`-display none`	`startvm --type headless`
`"label": "<text>"`	UI label only	UI label only
`"memory": 64…32768` MB	`-m <memory>`	`modifyvm --memory <memory>`
`"cpus": 1…64`	`-smp <cpus>`	`modifyvm --cpus <cpus>`
`"clock": "utc"`	`-rtc base=utc`	`modifyvm --rtcuseutc on`
`"clock": "localtime"`	`-rtc base=localtime`	`modifyvm --rtcuseutc off`
`"startupDiskEnabled": false`	No startup ISO or CD-ROM attachment	Same
`"startupDiskEnabled": true`	`GENISOIMAGE`, then `-drive file=<startupIso>,if=ide,index=2,media=cdrom,readonly=on,format=raw`	`GENISOIMAGE`, then `storageattach ... --type dvddrive --medium <startupIso>`
`"firmware": "bios"`	QEMU default BIOS; no flag	`modifyvm --firmware bios`
`"acceleration:qemu": "kvm"`	`-accel kvm`	—
`"acceleration:vbox": "default"`	—	No flag; VBox chooses its default execution engine
`"boot": "disk"`	`-boot order=c`	`modifyvm --boot1 disk --boot2 dvd --boot3 none --boot4 none`
`"cdromInterface": "ide"`	Startup `-drive ... if=ide`	`storagectl ... --add ide --controller PIIX4`
`"diskInterface:qemu": "ide"`	Working `-drive ... if=ide`	—
`"diskInterface:qemu": "virtio"`	Working `-drive ... if=virtio`	—
`"diskInterface:vbox": "sata"`	—	`storagectl ... --add sata --controller IntelAhci`
`"networkModel": "virtio"`	`-device virtio-net-pci,...`	`--nictype<N> virtio`
`"networkModel": "e1000"`	`-device e1000,...`	`--nictype<N> 82540EM`
`"image:qemu": "<raw path>"`	`QEMU_IMG create -f qcow2 -F raw -b <image> <workingQcow2>`	—
`"image:vbox": "<vdi path>"`	—	`createmedium disk --diffparent <image> --format VDI`
`"extraOptions:qemu": ["-machine", "q35", "-cpu", "host", "-nic", "none"]`	Insert the array as separate launch arguments; current profile supplies `-machine q35 -cpu host -nic none`	—

For example, `"memory:qemu": 1024` and `"memory:vbox": 2048` select the corresponding row's command with different values. `networkModel:qemu` and `networkModel:vbox` can likewise select different values. The runner supplies serial/monitor sockets, VDE cabling, disk paths and controller constants shown in the command sequences below; those values are derived from the host and switch records rather than extra system keywords.

`host.json`

Keyword	Values / validation	Default	Flag
`schemaVersion`	`{3}`	`3`	F
`id`	`h-` + 16 lowercase hex characters	generated	F
`name`	unique, 1–64 characters, no controls	generated	E
`system`	declared system ID	—	F
`engine`	`{qemu, vbox}`, same as session	session engine	F
`profileHash`	SHA-256 of selected resolved profile	generated	F
`config`	sparse editable overrides	`{}`	E
`config.memory`	integer 64–32768 MB	inherited	E
`config.cpus`	integer 1–64	inherited	E
`config.clock`	`{utc, localtime}`	inherited	E
`config.startupDiskEnabled`	`{false, true}`	inherited	E
`network.interfaces`	ordered list of 0–4 Ethernet entries	one disconnected	E
`network.interfaces[].switchId`	same-session switch ID or `null`	`null`	E
`network.interfaces[].port`	1–32 if connected, else `null`	allocated	G
`network.interfaces[].mac`	unique local unicast MAC	allocated	G

`host.json.config` stores only values different from the selected engine's system defaults. Restoring a default removes the override. The hash covers the resolved profile, with keys sorted lexicographically and serialized as compact UTF-8 JSON, excluding host overrides and `basedOn`. A profile change blocks host start and ZIP import until the original catalogue is restored or the host is recreated. Saved hardware and cable changes take effect on the next Play/Reboot/Reset, not by hotplug.

`switch.json`

Keyword	Values / validation	Default	Flag
`schemaVersion`	`{2}`	`2`	F
`id`	`s-` + 16 lowercase hex characters	generated	F
`name`	unique, 1–64 characters	generated	E
`kind`	`{vde}`	`vde`	F

Quick reference — files

File	Format and purpose	ZIP
`config.json`	UTF-8 JSON system catalogue	no
`.env`, `.env.local`	`NAME=value` executable paths	no
`images/<system>.raw`	QEMU base RAW	no
`images/<system>.vdi`	VBox base VDI	no
`runtime/<session-id>/session.json`	UTF-8 JSON session index	yes
`<host>/host.json`	UTF-8 JSON host configuration	yes
`<switch>/switch.json`	UTF-8 JSON switch configuration	yes
`<host>/startup/start.sh`	editable startup script	if present
`<host>/startup/<resource>`	startup companion file	if present
`<host>/disk.qcow2`	sparse QEMU working disk	optional
`<host>/disk.vdi`	VBox differential disk	no
`<host>/startup-<generation>.iso`	generated ISO 9660 startup disk	no
`<host>/serial.sock`, `<host>/mon.sock`	QEMU Unix sockets	no
`<switch>/mgmt.sock`, `<switch>/vde/`	VDE switch sockets	no

QEMU/KVM runner translation

The runner chooses `config.json.systems[host.system]`, resolves `:qemu` variants, checks `profileHash`, then applies `host.json.config` overrides. `N` is a zero-based Ethernet index. Connected switches start first. Executable placeholders come from `.env` / `.env.local` and command preflight.

```
<VDE_SWITCH> -s <vdeDir> -M <mgmtSocket>
<VDETERM> <mgmtSocketName>                # cwd=<switchDir>

# If <startupDiskEnabled>: stage startup/, substitute
# @HOSTNAME@, @IFNAMEn@, @SWPORTn@, then create ISO.
<GENISOIMAGE> -quiet -R -J -V QWEBSTART
  -o <startupIso> <isoStageDir>

# On first Play or Reset; Resume keeps the existing overlay.
<QEMU_IMG> create -f qcow2 -F raw
  -b <image:qemu> <workingQcow2>

<QEMU_SYSTEM_X86_64>
  <extraOptions:qemu> -accel <acceleration:qemu>
  -display none                           # mode=text
  -m <memory> -smp <cpus>
  -rtc base=<clock>
  -monitor unix:<monitorSocket>,server=on,wait=off
  -serial unix:<serialSocket>,server=on,wait=off
  -drive file=<workingQcow2>,if=<diskInterface:qemu>,format=qcow2,media=disk
  -boot order=c
  [-drive file=<startupIso>,if=<cdromInterface>,index=2,media=cdrom,readonly=on,format=raw]
  [-device <networkModel:qemu>,mac=<network.mac[N]>]  # disconnected
  [-netdev vde,id=lan<N>,sock=<vdeDir>,port=<network.port[N]>
   -device <networkModel:qemu>,netdev=lan<N>,mac=<network.mac[N]>]
```

`extraOptions:qemu` supplies `-machine q35 -cpu host -nic none` as separate argv tokens. `networkModel=virtio` translates to QEMU `virtio-net-pci`; `e1000` translates to `e1000`. `boot=disk` translates to `-boot order=c`. The fixed `firmware=bios` uses QEMU's default BIOS. `mode=text` translates to `-display none`. `<workingQcow2>`, `<startupIso>`, `<monitorSocket>` and `<serialSocket>` are derived under the host runtime directory; `<vdeDir>` and `<mgmtSocket>` are derived under the switch directory. `network.mac[N]` and `network.port[N]` come from `host.json.network.interfaces[N]`. KVM availability is checked before QEMU launch.

VirtualBox runner translation

The same profile resolution applies with `:vbox`. `N` is the one-based VBox adapter number (Ethernet index + 1). `acceleration:vbox=default` emits no VBoxManage flag.

```
<VBOXMANAGE> createvm --name <vmName> --basefolder <vmDir>
  --ostype Linux_64 --register
<VBOXMANAGE> modifyvm <vmName>
  --memory <memory> --cpus <cpus> --rtcuseutc <clockAsOnOff>
  --firmware <firmware>
  --nic<N> null --nictype<N> <networkModel:vbox>
    --macaddress<N> <network.mac[N]> --cableconnected<N> on
  # or, if connected:
  --nic<N> generic --nictype<N> <networkModel:vbox>
    --macaddress<N> <network.mac[N]> --cableconnected<N> on
    --nicgenericdrv<N> VDE
    --nicproperty<N> network=<vdeDir>[<network.port[N]>]
  --uart1 0x3F8 4 --uartmode1 server <serialSocket>
<VBOXMANAGE> createmedium disk --filename <workingVdi>
  --diffparent <image:vbox> --format VDI
<VBOXMANAGE> storagectl <vmName> --name SATA
  --add <diskInterface:vbox> --controller IntelAhci
<VBOXMANAGE> storageattach <vmName> --storagectl SATA
  --port 0 --device 0 --type hdd --medium <workingVdi>
<VBOXMANAGE> storagectl <vmName> --name 'Guest setup'
  --add <cdromInterface> --controller PIIX4
[<VBOXMANAGE> storageattach <vmName> --storagectl 'Guest setup'
  --port 1 --device 0 --type dvddrive --medium <startupIso>]
<VBOXMANAGE> modifyvm <vmName>
  --boot1 disk --boot2 dvd --boot3 none --boot4 none
<VBOXMANAGE> startvm <vmName> --type headless
```

`networkModel=virtio` translates to VBox `virtio`; `e1000` to `82540EM`. `clock=utc` maps to `--rtcuseutc on`, `localtime` to `off`. `boot=disk` sets the first boot device to disk. A disconnected card uses `--nic<N> null`; an absent card uses `none`. The working VDI and VM registration are kept for Resume/Reboot. VBox working disks are currently excluded from session ZIP files.

Executable configuration — `.env` and `.env.local`

Both files use one `NAME=value` assignment per line; blank lines and lines beginning with `#` are ignored. A command value is an executable name searched in `PATH` or an absolute path. The precedence is nonempty `.env.local` value, then process environment, then the required tracked `.env` default. `.env.local` is machine-specific and excluded from Git and session ZIPs. All eight keys are required in `.env` and checked at server startup.

Key	Runner use
`VBOXMANAGE`	VirtualBox management and VM launch
`GENISOIMAGE`	startup ISO generation
`NODE`	Node.js command preflight
`PYTHON3`	Python 3 command preflight
`QEMU_IMG`	QEMU working overlay creation
`QEMU_SYSTEM_X86_64`	QEMU VM launch
`VDE_SWITCH`	VDE switch launch
`VDETERM`	switch console

`PORT` is a process variable for the HTTP port; `QEMUWEB_TEST_SERVER` is reserved for integration fixtures. Neither is a system hardware keyword.

Web interface and session loading

Add a new host and **Host Settings** use the same tabs. General shows the host name, system, engine and base image. Hardware shows the resolved, read-only `type`, `arch`, `mode`, `firmware`, `acceleration` and `boot` values above editable memory, CPU and clock settings. Storage shows the read-only **Disk interface** and generated disk paths. Network shows the read-only **Network model** above Ethernet cabling. Startup begins with the read-only **CD-ROM interface**, then shows template values, `start.sh`, the startup disk option and ISO path. Expert shows console paths, the selected engine's **Extra options** (or `None`), and the full launch command preview. These read-only values come from the selected system/engine profile after inheritance; they are not additional host overrides.

Load session ZIP requires an empty current session. If any host or switch remains, a Warning dialog explains that all hosts and switches must be removed before choosing an archive. The server also rejects an import into a nonempty session with HTTP 409. This check prevents an archive from replacing current session contents silently.

Complete JSON examples and lifecycle

The maintained examples are `config.json`, `session.json`, `host.json` and `switch.json`. The host sample uses `debian13-legacy` on QEMU and overrides its 1024 MB default to 2048 MB. Its `profileHash` is computed from the resolved QEMU profile. `session.json.baseImages` is populated with image format, size and SHA-256 on ZIP export. A ZIP stores the session directory structure, excluding generated media, sockets, processes and RAM. QEMU working disks are included only when requested.

Changing `config.json` does not silently change an existing host: its stored hash must still match. `host.json.config` stores editable choices; `startup/start.sh` stores script text, not an inline JSON field. The runtime session directory is portable with respect to host and switch IDs, but its referenced base images must match on archive import.